

1.4 控制流

语句一般是顺序执行的：语句块的第一条语句首先执行，然后是第二条语句，依此类推。当然，少数程序，包括我们解决书店问题的程序，都可以写成只有顺序执行的形式。但程序设计语言提供了多种不同的控制流语句，允许我们写出更为复杂的执行路径。

1.4.1 while 语句

while 语句反复执行一段代码，直至给定条件为假为止。我们可以用 **while** 语句编写一段程序，求 1 到 10 这 10 个数之和：

```
#include <iostream>
int main()
{
    int sum = 0, val = 1;
    // 只要 val 的值小于等于 10, while 循环就会持续执行
    while (val <= 10) {
        sum += val; // 将 sum + val 赋予 sum
        ++val;      // 将 val 加 1
    }
    std::cout << "Sum of 1 to 10 inclusive is "
               << sum << std::endl;
    return 0;
}
```

我们编译并执行这个程序，它会打印出

Sum of 1 to 10 inclusive is 55

与之前的例子一样，我们首先包含头文件 `iostream`，然后定义 `main`。在 `main` 中我们定义两个 `int` 变量：`sum` 用来保存和；`val` 用来表示从 1 到 10 的每个数。我们将 `sum` 的初值设置为 0，`val` 从 1 开始。

12

这个程序的新内容是 **while** 语句。**while** 语句的形式为

```
while (condition)
    statement
```

while 语句的执行过程是交替地检测 *condition* 条件和执行关联的语句 *statement*，直至 *condition* 为假时停止。所谓条件（*condition*）就是一个产生真或假的结果的表达式。只要 *condition* 为真，*statement* 就会被执行。当执行完 *statement*，会再次检测 *condition*。如果 *condition* 仍为真，*statement* 再次被执行。**while** 语句持续地交替检测 *condition* 和执行 *statement*，直至 *condition* 为假为止。

在本程序中，**while** 语句是这样的

```
// 只要 val 的值小于等于 10, while 循环就会持续执行
while (val <= 10) {
    sum += val; // 将 sum + val 赋予 sum
    ++val;      // 将 val 加 1
}
```

条件中使用了小于等于运算符（`<=`）来比较 `val` 的当前值和 10。只要 `val` 小于等于 10，条件即为真。如果条件为真，就执行 **while** 循环体。在本例中，循环体是由两条语句组

成的语句块:

```
{
    sum += val;      // 将 sum + val 赋予 sum
    ++val;          // 将 val 加 1
}
```

所谓语句块 (block), 就是用花括号包围的零条或多条语句的序列。语句块也是语句的一种, 在任何要求使用语句的地方都可以使用语句块。在本例中, 语句块的第一条语句使用了**复合赋值运算符** (**+=**)。此运算符将其右侧的运算对象加到左侧运算对象上, 将结果保存到左侧运算对象中。它本质上与一个加法结合一个**赋值** (assignment) 是相同的:

```
sum = sum + val; // 将 sum + val 赋予 sum
```

因此, 语句块中第一条语句将 val 的值加到当前和 sum 上, 并将结果保存在 sum 中。

下一条语句

```
++val; // 将 val 加 1
```

使用**前缀递增运算符** (**++**)。递增运算符将运算对象的值增加 1。**++val** 等价于 **val=val+1**。

执行完 while 循环体后, 循环会再次对条件进行求值。如果 val 的值 (现在已经增加了) 仍然小于等于 10, 则 while 的循环体会再次执行。循环连续检测条件、执行循环体, 直至 val 不再小于等于 10 为止。

一旦 val 大于 10, 程序跳出 while 循环, 继续执行 while 之后的语句。在本例中, 继续执行打印输出语句, 然后执行 return 语句完成 main 程序。

1.4.1 节练习

13

练习 1.9: 编写程序, 使用 while 循环将 50 到 100 的整数相加。

练习 1.10: 除了++运算符将运算对象的值增加 1 之外, 还有一个递减运算符 (—) 实现将值减少 1。编写程序, 使用递减运算符在循环中按递减顺序打印出 10 到 0 之间的整数。

练习 1.11: 编写程序, 提示用户输入两个整数, 打印出这两个整数所指定的范围内的所有整数。

1.4.2 for 语句

在我们的 while 循环例子中, 使用了变量 val 来控制循环执行次数。我们在循环条件中检测 val 的值, 在 while 循环体中将 val 递增。

这种在循环条件中检测变量、在循环体中递增变量的模式使用非常频繁, 以至于 C++ 语言专门定义了第二种循环语句——**for 语句**, 来简化符合这种模式的语句。可以用 for 语句来重写从 1 加到 10 的程序:

```
#include <iostream>
int main()
{
    int sum = 0;
    // 从 1 加到 10
    for (int val = 1; val <= 10; ++val)
```

```

        sum += val; // 等价于 sum = sum + val
    std::cout << "Sum of 1 to 10 inclusive is "
        << sum << std::endl;
    return 0;
}

```

与之前一样，我们定义了变量 `sum`，并将其初始化为 0。在此版本中，`val` 的定义是 `for` 语句的一部分：

```

for (int val = 1; val <= 10; ++val)
    sum += val;

```

每个 `for` 语句都包含两部分：循环头和循环体。循环头控制循环体的执行次数，它由三部分组成：一个初始化语句 (*init-statement*)、一个循环条件 (*condition*) 以及一个表达式 (*expression*)。在本例中，初始化语句为

```
int val = 1
```

它定义了一个名为 `val` 的 `int` 型对象，并为其赋初值 1。变量 `val` 仅在 `for` 循环内部存在，在循环结束之后是不能使用的。初始化语句只在 `for` 循环入口处执行一次。循环条件

```
val <= 10
```

14 比较 `val` 的值和 10。循环体每次执行前都会先检查循环条件。只要 `val` 小于等于 10，就会执行 `for` 循环体。表达式在 `for` 循环体之后执行。在本例中，表达式

```
++val
```

使用前缀递增运算符将 `val` 的值增加 1。执行完表达式后，`for` 语句重新检测循环条件。如果 `val` 的新值仍然小于等于 10，就再次执行 `for` 循环体。执行完循环体后，再次将 `val` 的值增加 1。循环持续这一过程直至循环条件为假。

在此循环中，`for` 循环体执行加法

```
sum += val; // 等价于 sum = sum + val
```

简要重述一下 `for` 循环的总体执行流程：

1. 创建变量 `val`，将其初始化为 1。
2. 检测 `val` 是否小于等于 10。若检测成功，执行 `for` 循环体。若失败，退出循环，继续执行 `for` 循环体之后的第一条语句。
3. 将 `val` 的值增加 1。
4. 重复第 2 步中的条件检测，只要条件为真就继续执行剩余步骤。

1.4.2 节练习

练习 1.12: 下面的 `for` 循环完成了什么功能？`sum` 的终值是多少？

```

int sum = 0;
for (int i = -100; i <= 100; ++i)
    sum += i;

```

练习 1.13: 使用 `for` 循环重做 1.4.1 节中的所有练习（第 11 页）。

练习 1.14: 对比 for 循环和 while 循环，两种形式的优缺点各是什么？

练习 1.15: 编写程序，包含第 14 页“再探编译”中讨论的常见错误。熟悉编译器生成的错误信息。

1.4.3 读取数量不定的输入数据

在前一节中，我们编写程序实现了 1 到 10 这 10 个整数求和。扩展此程序一个很自然的方向是实现对用户输入的一组数求和。在这种情况下，我们预先不知道要对多少个数字求和，这就需要不断读取数据直至没有新的输入为止：

```
#include <iostream>
int main()
{
    int sum = 0, value = 0;
    // 读取数据直到遇到文件尾，计算所有读入的值的和
    while (std::cin >> value)
        sum += value; // 等价于 sum = sum + value
    std::cout << "Sum is: " << sum << std::endl;
    return 0;
}
```

15

如果我们输入

3 4 5 6

则程序会输出

Sum is: 18

main 的首行定义了两个名为 sum 和 value 的 int 变量，均初始化为 0。我们使用 value 保存用户输入的每个数，数据读取操作是在 while 的循环条件中完成的：

```
while (std::cin >> value)
```

while 循环条件的求值就是执行表达式

```
std::cin >> value
```

此表达式从标准输入读取下一个数，保存在 value 中。输入运算符（参见 1.2 节，第 7 页）返回其左侧运算对象，在本例中是 std::cin。因此，此循环条件实际上检测的是 std::cin。

当我们使用一个 istream 对象作为条件时，其效果是检测流的状态。如果流是有效的，即流未遇到错误，那么检测成功。当遇到文件结束符（end-of-file），或遇到一个无效输入时（例如读入的值不是一个整数），istream 对象的状态会变为无效。处于无效状态的 istream 对象会使条件变为假。

因此，我们的 while 循环会一直执行直至遇到文件结束符（或输入错误）。while 循环体使用复合赋值运算符将当前值加到 sum 上。一旦条件失败，while 循环将会结束。我们将执行下一条语句，打印 sum 的值和一个 endl。

从键盘输入文件结束符

当从键盘向程序输入数据时,对于如何指出文件结束,不同操作系统有不同的约定。在 Windows 系统中,输入文件结束符的方法是敲 Ctrl+Z (按住 Ctrl 键的同时按 Z 键),然后按 Enter 或 Return 键。在 UNIX 系统中,包括 Mac OS X 系统中,文件结束符输入是用 Ctrl+D。

16

再探编译

编译器的一部分工作是寻找程序文本中的错误。编译器没有能力检查一个程序是否按照其作者的意图工作,但可以检查形式 (form) 上的错误。下面列出了一些最常见的编译器可以检查出的错误。

语法错误 (syntax error): 程序员犯了 C++ 语言文法上的错误。下面程序展示了一些常见的语法错误; 每条注释描述了下一行中语句存在的错误:

```
// 错误: main 的参数列表漏掉了
int main ( {
    // 错误: endl 后使用了冒号而非分号
    std::cout << "Read each file." << std::endl:
    // 错误: 字符串字面常量的两侧漏掉了引号
    std::cout << Update master. << std::endl;
    // 错误: 漏掉了第二个输出运算符
    std::cout << "Write new master." std::endl;
    // 错误: return 语句漏掉了分号
    return 0
}
```

类型错误 (type error): C++ 中每个数据项都有其类型。例如, 10 的类型是 int (或者更通俗地说, “10 是一个 int 型数据”)。单词 “hello”, 包括两侧的双引号标记, 则是一个字符串字面值常量。一个类型错误的例子是, 向一个期望参数为 int 的函数传递了一个字符串字面值常量。

声明错误 (declaration error): C++ 程序中的每个名字都要先声明后使用。名字声明失败通常会导致一条错误信息。两种常见的声明错误是: 对来自标准库的名字忘记使用 std::、标识符名字拼写错误:

```
#include <iostream>
int main()
{
    int v1 = 0, v2 = 0;
    std::cin >> v >> v2; // 错误: 使用了 "v" 而非 "v1"
    // 错误: cout 未定义; 应该是 std::cout
    cout << v1 + v2 << std::endl;
    return 0;
}
```

错误信息通常包含一个行号和一条简短描述, 描述了编译器认为的我们所犯的错误。按照报告的顺序来逐个修正错误, 是一种好习惯。因为一个单个错误常常会具有传递效应, 导致编译器在其后报告比实际数量多得多的错误信息。另一个好习惯是在每修

正一个错误后就立即重新编译代码，或者最多是修正了一小部分明显的错误后就重新编译。这就是所谓的“编辑-编译-调试”(edit-compile-debug)周期。

1.4.3 节练习

17

练习 1.16: 编写程序，从 cin 读取一组数，输出其和。

1.4.4 if 语句

与大多数语言一样，C++也提供了 **if** 语句来支持条件执行。我们可以用 if 语句写一个程序，来统计在输入中每个值连续出现了多少次：

```
#include <iostream>
int main()
{
    // currVal 是我们正在统计的数；我们将读入的新值存入 val
    int currVal = 0, val = 0;
    // 读取第一个数，并确保确实有数据可以处理
    if (std::cin >> currVal) {
        int cnt = 1; // 保存我们正在处理的当前值的个数
        while (std::cin >> val) { // 读取剩余的数
            if (val == currVal) // 如果值相同
                ++cnt; // 将 cnt 加 1
            else { // 否则，打印前一个值的个数
                std::cout << currVal << " occurs "
                    << cnt << " times" << std::endl;
                currVal = val; // 记住新值
                cnt = 1; // 重置计数器
            }
        } // while 循环在这里结束
        // 记住打印文件中最后一个值的个数
        std::cout << currVal << " occurs "
            << cnt << " times" << std::endl;
    } // 最外层的 if 语句在这里结束
    return 0;
}
```

如果我们输入如下内容：

```
42 42 42 42 42 55 55 62 100 100 100
```

则输出应该是：

```
42 occurs 5 times
55 occurs 2 times
62 occurs 1 times
100 occurs 3 times
```

有了之前多个程序的基础，你对这个程序中的大部分代码应该比较熟悉了。程序以两个变量 val 和 currVal 的定义开始：currVal 记录我们正在统计出现次数的那个数；val 则保存从输入读取的每个数。与之前的程序相比，新的内容就是两个 if 语句。第一条 if 语句


```
18 if (std::cin >> currVal) {
    // ...
} //最外层的 if 语句在这里结束
```

保证输入不为空。与 while 语句类似，if 也对一个条件进行求值。第一条 if 语句的条件是读取一个数值存入 currVal 中。如果读取成功，则条件为真，我们继续执行条件之后的语句块。该语句块以左花括号开始，以 return 语句之前的右花括号结束。

如果需要统计出现次数的值，我们就定义 cnt，用来统计每个数值连续出现的次数。与上一小节的程序类似，我们用一个 while 循环反复从标准输入读取整数。

while 的循环体是一个语句块，它包含了第二条 if 语句：

```
if (val == currVal)           // 如果值相同
    ++cnt;                    // 将 cnt 加 1
else {                        // 否则，打印前一个值的个数
    std::cout << currVal << " occurs "
                << cnt << " times" << std::endl;
    currVal = val;            // 记住新值
    cnt = 1;                  // 重置计数器
}
```

这条 if 语句中的条件使用了**相等运算符**（==）来检测 val 是否等于 currVal。如果是，我们执行紧跟在条件之后的语句。这条语句将 cnt 增加 1，表明我们再次看到了 currVal。

如果条件为假，即 val 不等于 currVal，则执行 else 之后的语句。这条语句是一个由一条输出语句和两条赋值语句组成的语句块。输出语句打印我们刚刚统计完的值的出现次数。赋值语句将 cnt 重置为 1，将 currVal 重置为刚刚读入的值 val。



C++用=进行赋值，用==作为相等运算符。两个运算符都可以出现在条件中。一个常见的错误是想在条件中使用==（相等判断），却误用了=。

1.4.4 节练习

练习 1.17：如果输入的所有值都是相等的，本节的程序会输出什么？如果没有重复值，输出又会是怎样的？

练习 1.18：编译并运行本节的程序，给它输入全都相等的值。再次运行程序，输入没有重复的值。

练习 1.19：修改你为 1.4.1 节练习 1.10（第 11 页）所编写的程序（打印一个范围内的数），使其能处理用户输入的第一个数比第二个数小的情况。

关键概念：C++程序的缩进和格式

C++程序很大程度上是格式自由的，也就是说，我们在哪里放置花括号、缩进、注释以及换行符通常不会影响程序的语义。例如，花括号表示 main 函数体的开始，它可以放在 main 的同一行中；也可以像我们所做的那样，放在下一行的起始位置；还可以放在我们喜欢的其他任何位置。唯一的要求是左花括号必须是 main 的形参列表后第一个非空、非注释的字符。

虽然很大程度上可以按照自己的意愿自由地设定程序的格式，但我们所做的选择会影响程序的可读性。例如，我们可以将整个 `main` 函数写在很长的单行内，虽然这样是合乎语法的，但会非常难读。

关于 C/C++ 的正确格式的辩论是无休止的。我们的信条是，不存在唯一正确的风格，但保持一致性是非常重要的。例如，大多数程序员都对程序的组成部分设置恰当的缩进，就像我们在之前的例子中对 `main` 函数中的语句和循环体所做的那样。对于作为函数界定符的花括号，我们习惯将其放在单独一行中。我们还习惯对复合 IO 表达式设置缩进，以使输入输出运算符排列整齐。其他一些缩进约定也都会令越来越复杂的程序更加清晰易读。

我们要牢记一件重要的事情：其他可能的程序格式总是存在的。当你要选择一种格式风格时，思考一下它会对程序的可读性和易理解性有什么影响，而一旦选择了一种风格，就要坚持使用。

1.5 类简介

在解决书店程序之前，我们还需要了解的唯一一个 C++ 特性，就是如何定义一个数据结构（`data structure`）来表示销售数据。在 C++ 中，我们通过定义一个类（`class`）来定义自己的数据结构。一个类定义了一个类型，以及与其关联的一组操作。类机制是 C++ 最重要的特性之一。实际上，C++ 最初的一个设计焦点就是能定义使用上像内置类型一样自然的类类型（`class type`）。

在本节中，我们将介绍一个在编写书店程序中会用到的简单的类。当我们在后续章节中学习了更多关于类型、表达式、语句和函数的知识后，会真正实现这个类。

为了使用类，我们需要了解三件事情：

- 类名是什么？
- 它是在哪里定义的？
- 它支持什么操作？

对于书店程序来说，我们假定类名为 `Sales_item`，头文件 `Sales_item.h` 中已经定义了这个类。

如前所见，为了使用标准库设施，我们必须包含相关的头文件。类似的，我们也需要使用头文件来访问为自己的应用程序所定义的类。习惯上，头文件根据其中定义的类的名字来命名。我们通常使用 `.h` 作为头文件的后缀，但也有一些程序员习惯 `.h`、`.hpp` 或 `.hxx`。标准库头文件通常不带后缀。编译器一般不关心头文件名的形式，但有的 IDE 对此有特定要求。

◀ 20

1.5.1 `Sales_item` 类

`Sales_item` 类的作用是表示一本书的总销售额、售出册数和平均售价。我们现在不关心这些数据如何存储、如何计算。为了使用一个类，我们不必关心它是如何实现的，只需知道类对象可以执行什么操作。

每个类实际上都定义了一个新的类型，其类型名就是类名。因此，我们的 `Sales_item` 类定义了一个名为 `Sales_item` 的类型。与内置类型一样，我们可以定义类类型的变量。当我们写下如下语句